

Phase -1.12 — Git Strategy & Development Workflow

Objective

Version control is far more than a backup system.

It provides:

- Change tracking
- Collaboration
- Code review
- Release management
- Continuous Integration
- Deployment automation

Every production project should define its Git workflow before development begins.

This phase establishes the repository standards that will be followed throughout the project.

Repository Structure

A single Git repository will be used.

```
seat-sync/  
|  
├─ backend/  
├─ frontend/  
├─ docs/  
├─ infrastructure/  
├─ scripts/  
└─ .github/
```

This is a **monorepo**.

Keeping everything together simplifies development while preserving clear boundaries between applications and infrastructure.

Branching Strategy

The project follows a simplified GitFlow model.

```
main
|
develop
|
├─ feature/auth
├─ feature/events
├─ feature/bookings
├─ feature/payments
├─ feature/cache
├─ feature/docker
└─ feature/observability
```

main

Purpose:

Production-ready code only.

Rules:

- Always deployable.
- Protected branch.
- Direct commits are prohibited.
- Changes only arrive through Pull Requests.

develop

Purpose:

Integration branch.

Every completed feature is merged here first.

After testing, changes are promoted to `main`.

Feature Branches

Every new feature receives its own branch.

Examples:

```
feature/auth  
  
feature/events  
  
feature/booking  
  
feature/payment  
  
feature/docker
```

Benefits:

- Isolated development.
- Easier reviews.
- Safer experimentation.
- Cleaner history.

Branch Naming Convention

```
feature/<feature-name>  
  
bugfix/<issue>  
  
hotfix/<issue>  
  
refactor/<module>  
  
docs/<topic>  
  
test/<module>  
  
chore/<task>
```

Examples:

```
feature/jwt-auth  
  
feature/redis-cache  
  
bugfix/seat-lock  
  
docs/architecture  
  
refactor/auth-service
```

Commit Strategy

Every commit should represent one logical unit of work.

Avoid:

- Mixing unrelated changes.
- Large "everything" commits.
- Committing broken code.

Conventional Commits

The project follows the Conventional Commits specification.

Examples:

```
feat(auth): implement JWT authentication  
  
fix(booking): prevent duplicate seat reservation  
  
refactor(cache): simplify Redis client  
  
docs(architecture): add sequence diagrams  
  
test(auth): add login integration tests  
  
chore(docker): update compose configuration  
  
ci(github): add lint workflow
```

Benefits include:

- Readable history.
 - Automatic changelogs.
 - Semantic versioning support.
 - Easier code reviews.
-

Pull Requests

Every feature should be merged through a Pull Request.

A good Pull Request includes:

- Description of the change.
- Screenshots (frontend, when applicable).
- Testing performed.
- Linked issue.
- Potential risks.

Even when working alone, this habit mirrors professional development practices.

Code Reviews

Before merging, review the following:

- Does the code solve the problem?
 - Is the architecture respected?
 - Are naming conventions followed?
 - Are tests included?
 - Is error handling complete?
 - Is logging appropriate?
 - Are edge cases handled?
 - Is documentation updated?
-

Merge Strategy

The project will use **Squash and Merge**.

Advantages:

- Cleaner Git history.

- One commit per completed feature.
 - Easier rollback.
 - Simpler release history.
-

Release Strategy

Stable milestones should be tagged.

Examples:

v0.1.0

v0.2.0

v1.0.0

Suggested milestones:

- v0.1.0 — Project Setup
 - v0.2.0 — Authentication
 - v0.3.0 — Event Management
 - v0.4.0 — Booking System
 - v0.5.0 — Payment Module
 - v0.6.0 — Redis & BullMQ
 - v0.7.0 — Observability
 - v0.8.0 — Docker & CI/CD
 - v1.0.0 — Production Ready
-

GitHub Actions

Every push should automatically execute:

Push

↓

Install Dependencies

↓

Lint

↓
Type Check
↓
Unit Tests
↓
Integration Tests
↓
Build
↓
Success / Failure

No broken code should be merged into `main`.

Issue Management

Every major task should begin with a GitHub Issue.

Issue categories:

- Feature
- Bug
- Enhancement
- Documentation
- Technical Debt

Issues become the source of truth for planned work.

Project Boards

Development should follow a Kanban workflow.

Backlog

↓

Ready

↓

In Progress

↓

Review

↓

Testing

↓

Done

GitHub Projects will be used to manage this workflow.

Repository Standards

The repository should always include:

README.md

LICENSE

.gitignore

.env.example

CONTRIBUTING.md

CODE_OF_CONDUCT.md

SECURITY.md

CHANGELOG.md

Not all files need to be completed immediately, but the structure should exist from the beginning.

Git Ignore Strategy

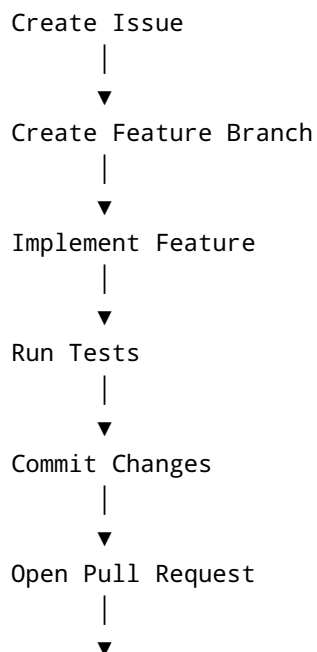
The following should never be committed:

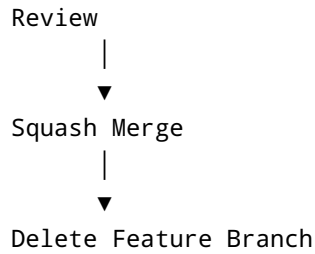
- .env
- node_modules
- Build artifacts
- Coverage reports
- IDE settings
- Log files
- Local databases
- Temporary files

Sensitive information must never enter Git history.

Development Workflow

Every feature follows the same lifecycle.





Following a consistent workflow reduces mistakes and makes collaboration significantly easier.

Deliverables

At the completion of this phase, the project has established:

- Repository structure.
- Branching strategy.
- Commit conventions.
- Pull request workflow.
- Code review checklist.
- Release strategy.
- GitHub Actions workflow.
- Repository standards.
- Development lifecycle.

These practices will be followed from the very first commit through the final production release.

The next phase, **Phase -1.13 — Coding Standards & Engineering Guidelines**, will define naming conventions, project organization, code style, testing standards, documentation practices, logging conventions, and architectural rules that every line of code must follow.